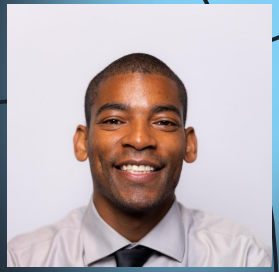
# Meet the MAYTHON Team


Fahim Tanvir



Jude Marryshow

Mosharoof Hossain

Labib Nafi

# Multivariate Cryptography & Oil–Vinegar (O&V)

**Multivariate Quadratic (MQ)** equations are **NP-hard**, even for quantum computers.

MQ cryptography shapes the basis for many post-quantum signature schemes.

The **Oil–Vinegar (O&V)** design divides variables into *oil* and *vinegar* groups.

Combining these variables creates a **trapdoor** that makes signing easy but inversion difficult.

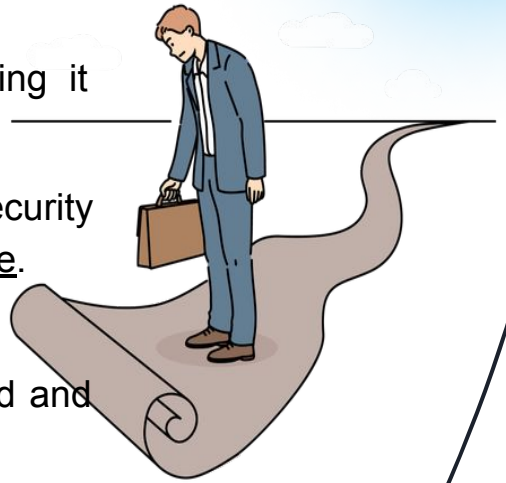
Provides **fast signing and verification** without depending on factoring.

# Limitations of Previous MQ Schemes

Prior multivariate schemes like Rainbow and UOV (Unbalanced Oil and Vinegar) faced several problems.

1. ***Large Key Sizes***
   - The public key could be *several hundred* kilobytes long, making it unsuitable for devices with limited memory.
   - Examples include smartphones, gaming consoles, and home security systems, which often run on low-power chips with restricted storage.

2. ***Structural Weaknesses***
   - Researchers discovered patterns in how these systems generated and used their equations.
   - Once analyzed, these patterns exposed parts of the private key, weakening the system's overall security.



# Limitations of Previous MQ Schemes

**Example - The Rainbow Attack**

- Rainbow was broken in 2022 when researchers found math dependencies in its design.
- These dependencies helped them to reconstruct the private key using only the public key, showing that the scheme's inner structure wasn't concealed well enough.

**Impact on Research**

- These weaknesses showed that better multivariate schemes were needed. Schemes that could:
  a. Conceal structural patterns better
  b. Decrease key size while preserving security, and
  c. Remain efficient for present-day systems.

# The Birth of MAYO

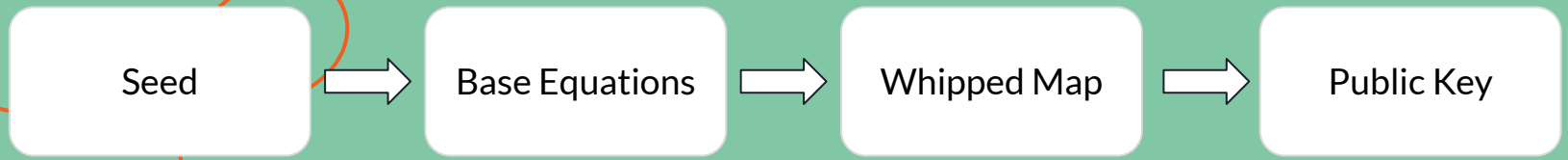
- Launches the “whipping” technique to compress keys.
- Uses a smaller oil space → less equations, smaller parameters.
- Keeps strong trapdoor structure for secure signatures.
- Balances compactness, effectiveness and stability.

***MAYO = Applicable, lightweight post-quantum digital signature scheme.***

# Scheme Overview

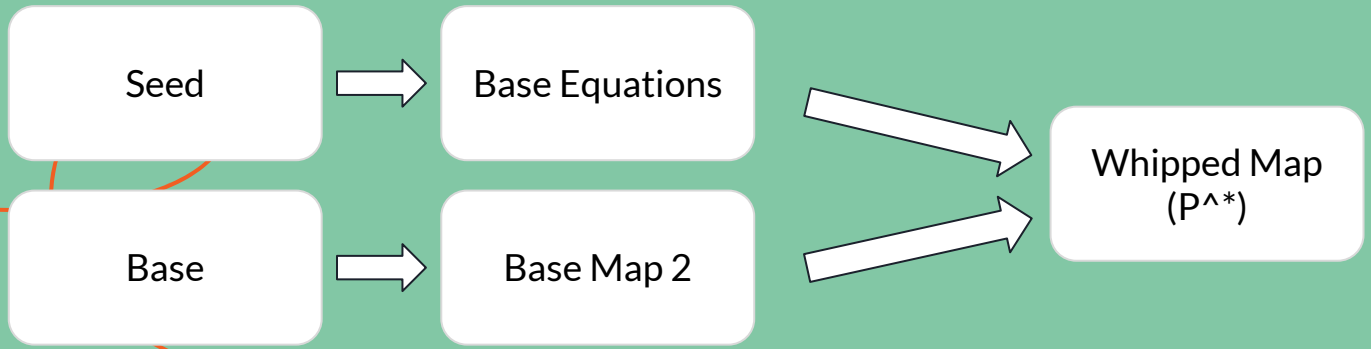
# MAYO at a Glance

- MAYO is based on UOV (Unbalanced Oil and Vinegar), a well-known multivariate signature scheme.
- The main problem with UOV: large key sizes (especially the public key).
- MAYO solves this by introducing whipped maps, a structural trick that lets us generate the key deterministically from a small seed.
- This means we can rebuild the full key whenever needed, instead of storing the entire matrix.
- Result: drastically smaller key sizes without hurting performance or security.



# The Whipped Map Construction

- Whipping = combining multiple base maps generated from the same seed.
- Each base map corresponds to an instance of the UOV structure.
- By using a *whipping parameter* **k**, we combine ***k*** instances of the map creating a composite public map P*.
- The result expands the **oil space dimension** (from *o* to *k times o*) and strengthens security.
- This composite mapping gives MAYO its **compactness** and **strong resistance** to algebraic attacks.



# System Parameters and Components

- $n$ = total number of variables (vinegar + oil)
- $m$ = number of equations
- $o$ = oil dimension (affects difficulty of solving system)
- $k$ = whipping parameter (number of maps combined)
- MAYO’s compactness comes from generating everything from a short random seed, not storing large matrices.
- This structure ensures deterministic key generation — both sides can recreate the same key from the same seed.

# How the Scheme Operates

**KeyGen:**

- Start from a random seed.
- Generate private affine transformations (S, T) and the public map P*.
- S: linear transformation applied before the central map.
- T: linear transformation applied after the central map.
- Output: Private key = seed + transformations, Public key = P*

**Sign:**

- Use private key to invert the central map efficiently In other words, they find a preimage under P* that corresponds to the given message hash.
- Produce a valid signature (vector) corresponding to the message hash.
- Unlike traditional schemes that rely on modular arithmetic of elliptic curves, this uses multivariate polynomial inversion as its core operation

# How the Scheme Operates

**Verify:**

- Anyone can use the public map P* to check if the signature satisfies the polynomial equations
- The verifier checks whether the given signature actually satisfies the public equations: P*(x)=hash(message)
- Only the public information is used no access to the seed or transformations.

**What's public vs. private:**

- Public: P*, message hash, signature
- Private: seed, affine transformations

# Algorithms

# Key Generation

**Algorithm 1** MAYO.KeyGen() [9]

**Output:** Public key $pk$, secret key $sk$

1: $\mathbf{O} \leftarrow \mathbb{F}_q^{o \times (n-o)}$
2: $\text{seed}_{sk} \leftarrow \{0,1\}^{\lambda}$
3: $\text{seed}_{pk} \leftarrow \text{SHAKE256}(\text{seed}_{sk})$
4: **for** $i$ from 1 to $m$ **do**
5: $\quad \mathbf{P}_i^{(1)} \leftarrow \text{Expand}(\text{seed}_{pk} \parallel P1 \parallel i)$
6: $\quad \mathbf{P}_i^{(2)} \leftarrow \text{Expand}(\text{seed}_{pk} \parallel P2 \parallel i)$
7: $\quad \mathbf{P}_i^{(3)} \leftarrow \text{Upper}(-\mathbf{O}\mathbf{P}_i^{(1)}\mathbf{O}^T - \mathbf{O}\mathbf{P}_i^{(2)})$
8: **return** $(pk, sk) = ((\text{seed}_{pk}, \{\mathbf{P}_i^{(3)}\}_{1 \le i \le m}), (\text{seed}_{sk}, \mathbf{O}))$

**1. (Choose) Secret**
Sample a random matrix ***O***. This is the core secret structure, used to simplify solving the system later.

**2. (Generate) Seeds**
Create *seedsk* (Secret) and derive *seedpk* (Public). Deterministically links the secret and public parts for consistency.

**3. (Build Public Key)**
Use *seedpk* to create matrices $P_i(1)$, $P_i(2)$. These are public constants defining the quadratic equation structure.

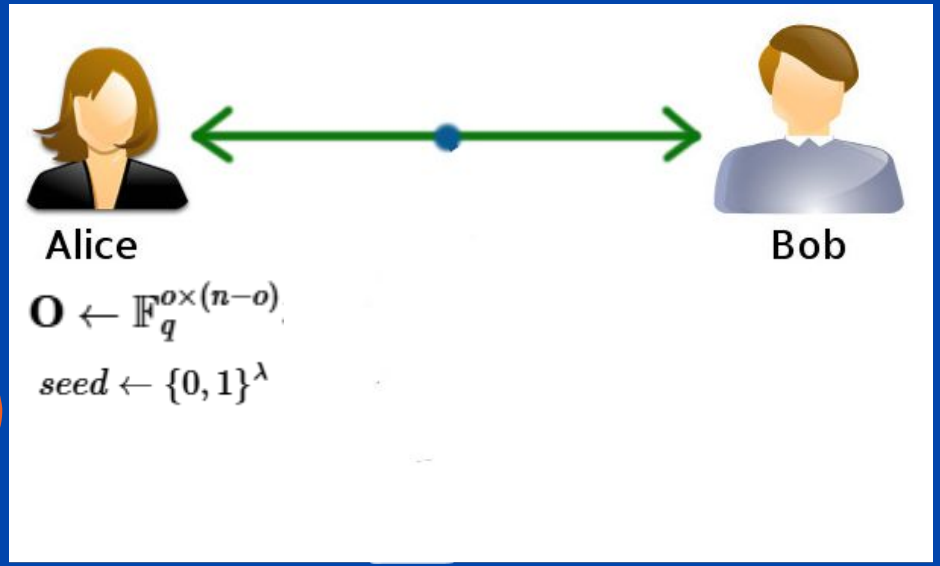
**4. (Combine)**
Compute the final public matrices $P_i(3)$ using ***O***, $P_i(1)$, $P_i(2)$. $P_i(3)$ represents the complex public quadratic system.

**Outputs:**
pk = (*seedpk*, $\{P_i(3)\}$)
sk = (*seedsk*, ***O***)

# Key Generation

**1. Create the Trapdoor**

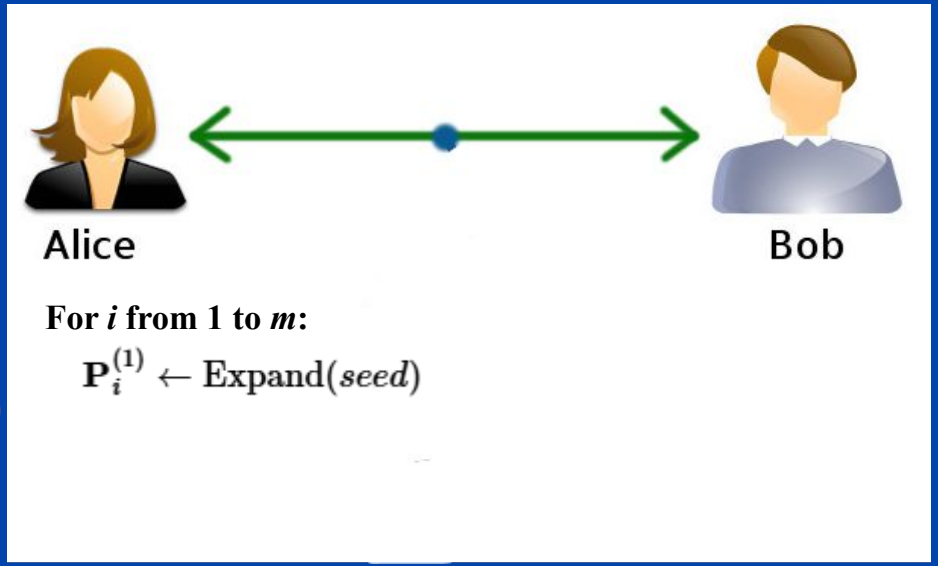
Alice creates a special system of quadratic equations (P) that is structured according to the original Oil and Vinegar design. ***O*** is her starting matrix where P will equal 0.



# Key Generation

**2. Create the Shufflers**

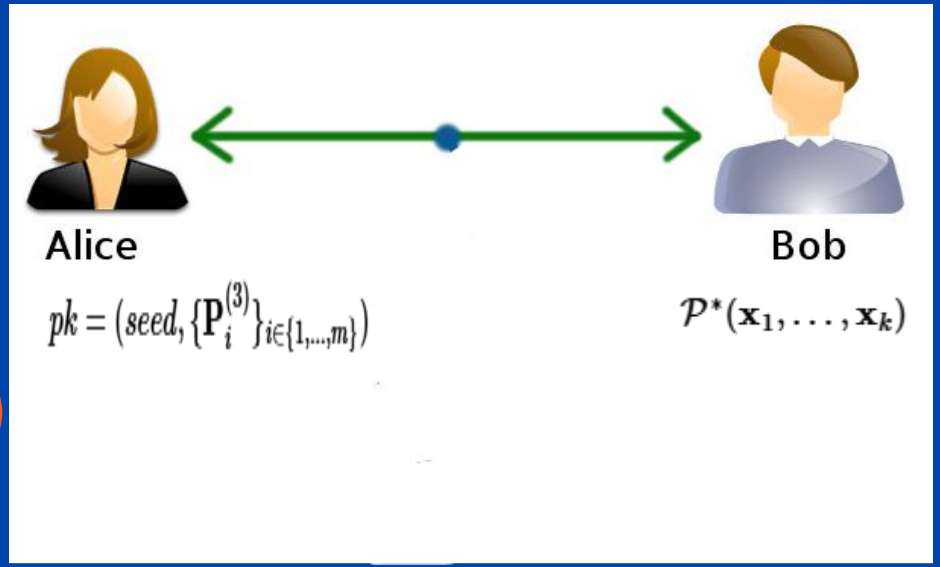
Alice chooses two random, invertible linear mixing functions, *S* and *T*. These are her random shuffling cards. She'll use them to completely hide the simple structure of the trapdoor P.



# Key Generation

**3. Build the Public Lock**

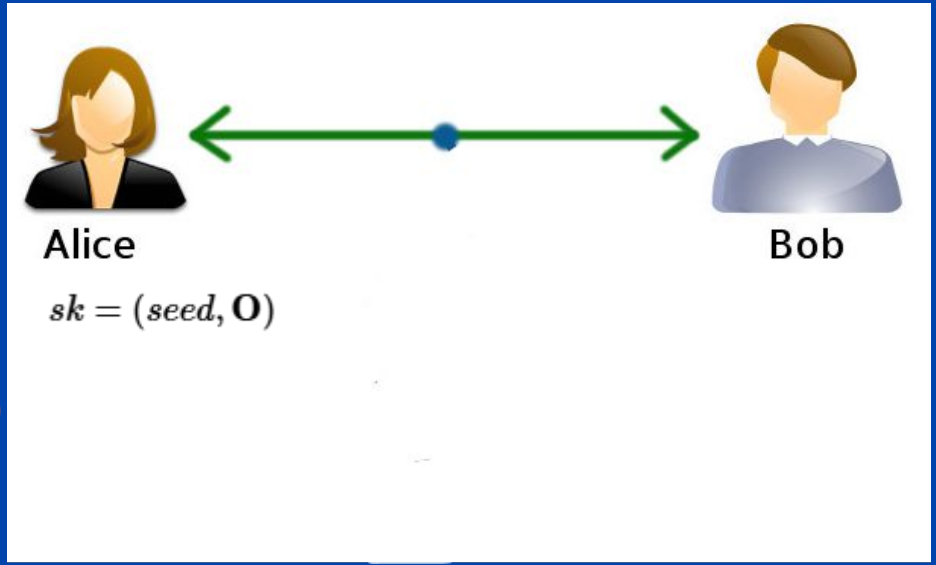
Alice composes the three functions:
$P* = S \circ P \circ T$. She then publishes the coefficients of this massive, complex $P*$ as her Public Key. This is the final, scrambled equation. It's so tangled that no one can easily see the structure inside.



# Key Generation

**4. Keep the Secret Key**

Alice keeps the original trapdoor P and the two shufflers (*S* and *T*) as her Secret Key. This is the key to creating a valid signature: the map P tells her how the equations are structured, and *S* and *T* tell her how to un-scramble and descramble everything.



# Sign

**Algorithm 2** MAYO.Sign(sk, $M$) [9]

**Input:** Secret key sk, message $M$
**Output:** Signature $\sigma$

1: $(\text{seed}_{sk}, \mathbf{O}) \leftarrow \text{sk}$
2: $\text{seed}_{pk} \leftarrow \text{SHAKE256}(\text{seed}_{sk})$
3: **for** $i$ from 1 to $m$ **do**
4: $\quad \mathbf{P}_i^{(1)} \leftarrow \text{Expand}(\text{seed}_{pk} \| P1 \| i)$
5: $\quad \mathbf{P}_i^{(2)} \leftarrow \text{Expand}(\text{seed}_{pk} \| P2 \| i)$
6: $R \leftarrow \{0,1\}^r$ ▷ *Deterministic variant:* $R \leftarrow \{0\}^r$
7: $\text{salt} \leftarrow \text{SHAKE256}(M \| R \| \text{seed}_{sk})$
8: $\mathbf{t} \leftarrow \text{SHAKE256}(M \| \text{salt})$
9: **for** $ctr$ from 0 to 255 **do**
10: $\quad \text{V} \leftarrow \text{SHAKE256}(M \| \text{salt} \| \text{seed}_{sk} \| ctr)$
11: $\quad \text{v}_1, \ldots, \text{v}_k \leftarrow \text{Decode}(\text{V})$
12: $\quad (\mathbf{A}, \mathbf{y}) \leftarrow \text{BuildLinearSystem}(\{\text{v}_1, \ldots, \text{v}_k\}, \mathbf{O}, \mathbf{P}^{(1)}, \mathbf{P}^{(2)}, \mathbf{t})$
13: $\quad \text{x} \leftarrow \text{SampleSolution}(\mathbf{A}, \mathbf{y})$ ▷ *Try to find* $Ax = y$ *(i.e.* $\mathcal{P}^*(\mathbf{s}) = \mathbf{t}$*)*
14: $\quad$ **if** $\text{x} \neq \perp$ **then break**
15: $\text{s} \leftarrow \{\text{v}_i + \mathbf{O}\text{x}_i \| \text{x}_i\}_{1 \le i \le k}$
16: **return** $\sigma = (\text{s}, \text{salt})$

**1. (Initialization)**
Re-derive public constants $P_i(1)$, $P_i(2)$ from *seedsk*
Ensures the signer works with the correct public system.

**2. (Target Hash)**
Compute salt and the target vector *t* = Hash(M // salt). *t* is the specific output the quadratic system must produce.

**3. (Build Linear System)**
Construct a solvable linear system *Ax* = *y* using the ***O*** matrix and *t*. This transforms the hard quadratic problem into an easy linear problem using the secret key ***O***.

**4. (Solve)**
Find the solution vector *x* for the linear system. The necessary component for the signature is found.

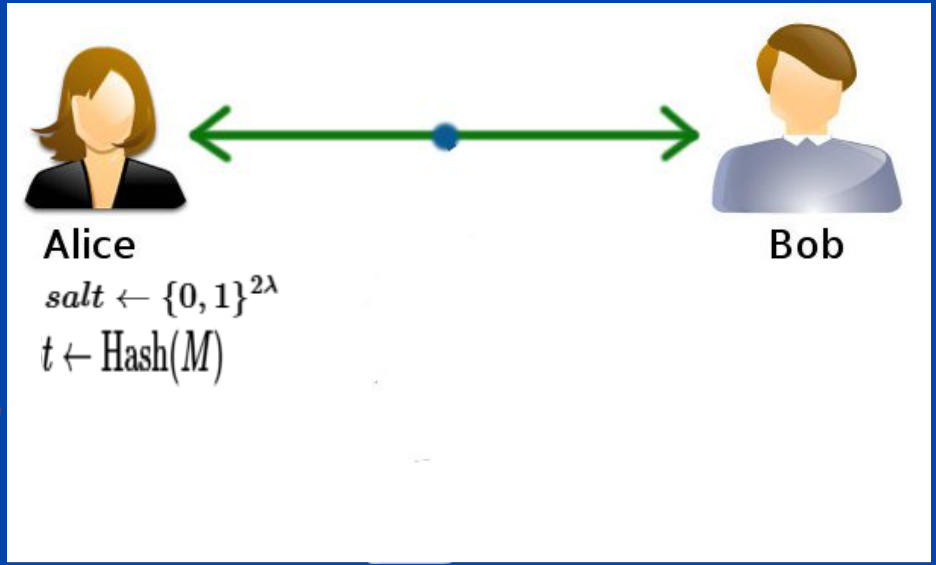
**5. (Finalize)**
Compute the final signature vector *s* from *x* and other components. *s* is the input vector that satisfies the public quadratic system.

**Output Signature** σ = (*s*, salt)

# Sign

**1. Define the Target**

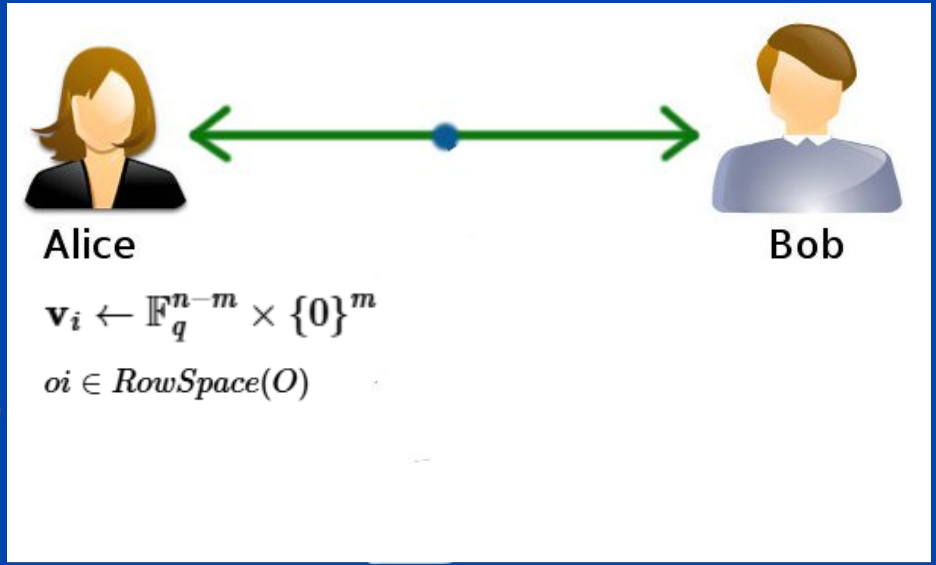
Alice first calculates the hash of the message: H($\mu$). This hash is her target output. This is the goal number she needs her final equation to equal.



# Sign

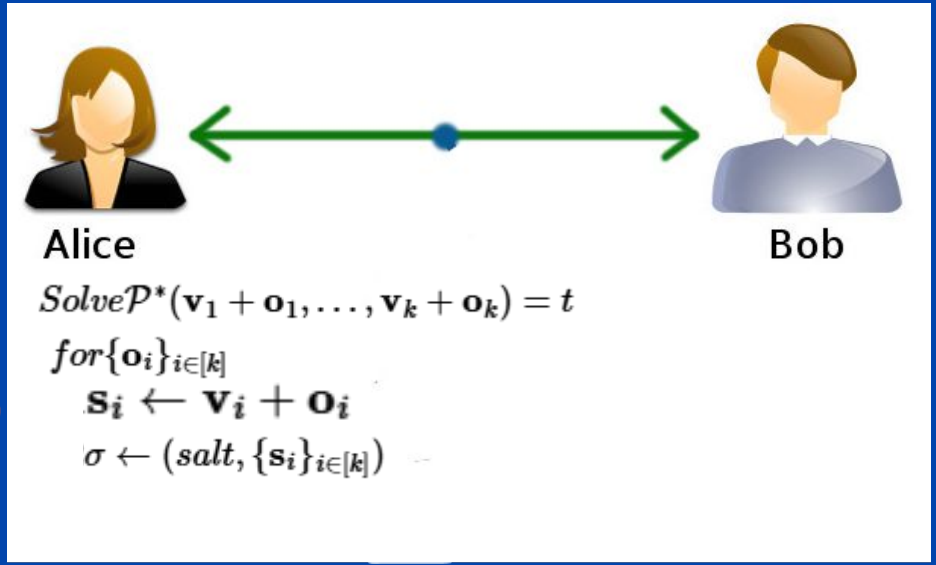
**2. Un-shuffle the Target**

Using her secret shuffler S, she calculates the inverse target $b = S^{-1}(H(\mu))$. She figures out what the target number must have been before she applied her output shuffler S. The goal is now to solve P(…) = $b$.



# Sign

**3. Solve the trapdoor**



# Verification

**Algorithm 3** MAYO.Verify(pk, $M$, $\sigma$) [9]

**Input:** Public key pk, message $M$, signature $\sigma$
**Output:** Boolean

1: $(\text{seed}_{pk}, \mathbf{P}^{(3)}) \leftarrow pk$
2: **for** $i$ from 1 to $m$ **do**
3: $\quad \mathbf{P}_i^{(1)} \leftarrow \text{Expand}(\text{seed}_{pk} \parallel P1 \parallel i)$
4: $\quad \mathbf{P}_i^{(2)} \leftarrow \text{Expand}(\text{seed}_{pk} \parallel P2 \parallel i)$
5: $(\mathbf{s}, \text{salt}) = \sigma$
6: $\mathbf{t} \leftarrow \text{SHAKE256}(M \parallel \text{salt})$
7: $\mathbf{t}' \leftarrow \text{Evaluate}_{\mathbf{P}^{(1)},\mathbf{P}^{(2)},\mathbf{P}^{(3)}}(\mathbf{s})$ ▷ $\mathcal{P}^*(\mathbf{s}) = \mathbf{t}'$
8: **return** true if $\mathbf{t} = \mathbf{t}'$ else false

**1. (Setup)**
Extract (s, salt) from σ. Re-derive $P_i(1)$, $P_i(2)$ from *seedpk*. Prepare the public components needed for the check.

**2. (Recalculate Target)**
Compute the expected target t′ = Hash(M // salt). Determines what the public system should output for this message/salt pair.

**3. (Evaluate)**
Plug the signature vector s into the public quadratic system defined by $P_i(1)$, $P_i(2)$, $P_i(3)$.This is the core check: Calculate the actual output t of the public system when using s.

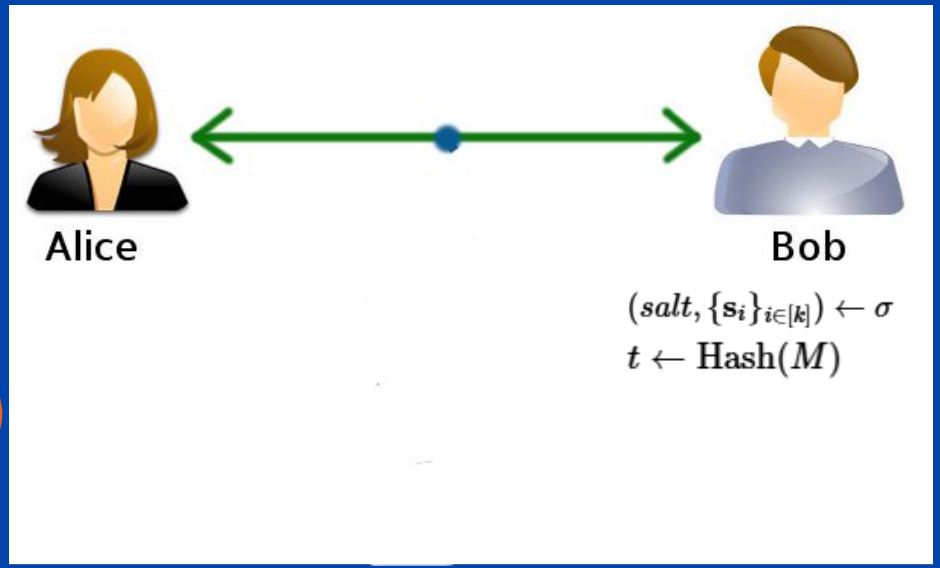
**4. (Compare)**
Check if t = t′. If they match, the signature is VALID. The input s truly leads to the expected hash t′.

**Output: TRUE** (Valid) or **FALSE** (Invalid)

# Verification

**1. Get the Target**

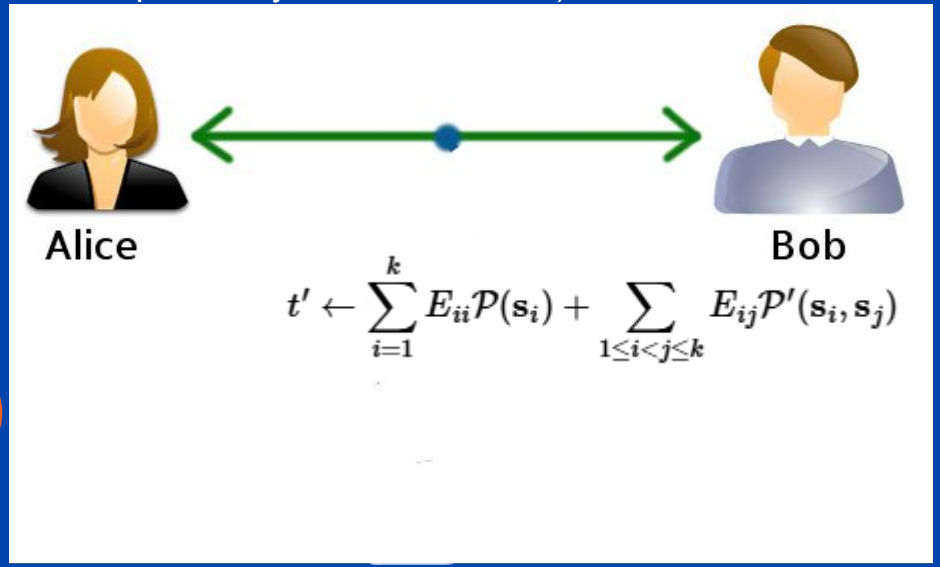
Bob calculates the hash of the message: H($\mu$). He determines the target number the equation should equal.



# Verification

**2. Test the Lock**

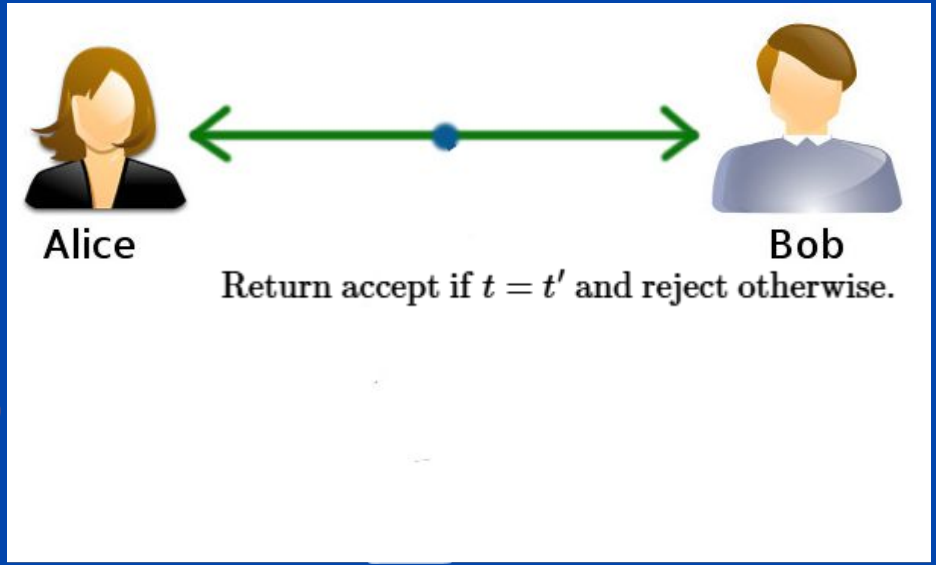
Bob takes the signature *s* and plugs it directly into Alice's massive, complex Public Key equation P. He calculates the output *y = P∗(s)*. He plugs the key (*s*) into the lock (P∗). Since this is a massive quadratic system, it takes him a bit of time to compute, but it's straightforward (no need to solve equations, just evaluate them).



# Verification

**3. Compare**

Bob checks: Is the output y equal to the target hash H($\mu$)? If the signature came from Alice (i.e., she used the secret trapdoor correctly), the output must match the hash.
Result: If $y = H(\mu)$, the signature is VALID. Otherwise, it's invalid.



# Security Analysis

# Security Analysis

- One of the many benefits of MAYO is its lightened computational load.
- This makes it quicker and easier to follow than something popular such as RSA, which falls victim due to its factorization techniques within its computations.
- However, to analyze its applications in security, we have to take a look at its older brother: the Oil and Vinegar scheme.

# Oil and Vinegar (OV) Problem

- The Oil and Vinegar signature scheme asks an adversary to distinguish a random multivariate quadratic map P from one that vanishes on the row space of a specific O.
- This problem is considered relatively well-understood. However it does have a caveat of rather large public keys. This is good for MAYO as it is just a variant of the OV problem with a reduced oil space.
- This means that MAYO is something that is computationally readily available and the hardness of distinguishing structured maps from random ones is preserved under this restriction.
- MAYO inherits the foundational security assumptions of OV while offering a more compact and implementation-friendly design.
- Moreover, the reduced oil space simplifies the algebraic structure of the public key, which can lead to performance gains in both signature generation and verification.
- This makes MAYO a promising candidate for constrained environments, such as embedded systems or post-quantum secure messaging protocols.

# MAYO’s Own Features

- The Multi-Target Whipped MQ Problem extends the OV scheme by introducing structured bilinear combinations across multiple inputs and targets, making it harder to forge signatures on average.
- Its older counterpart, OV focuses on distinguishing structured quadratic maps from random ones using a single trapdoor and target, which is effective but MQ is more secure.
- What sets Mayo apart is its emphasis on efficiency and adaptability. Although still in the testing phase, the scheme is being continuously refined to minimize memory consumption without compromising computational performance.
- This optimization is crucial for deployment in environments ranging from lightweight embedded systems to high-throughput quantum-resistant infrastructure.
- Mayo is engineered to maintain consistent performance across both conventional and quantum computing platforms.
- The signing process for it is designed to run in constant time to prevent information leaks from timing attacks, improves security.

# Limitations and Caveats of MAYO

- MAYO, while efficient and definitely faster & more optimized than something like RSA, still requires heavy computation and can occasionally fail to sign if its linear system lacks full rank.
- This is a rare but important design consideration, especially since it's still being tested.
- Also since the seed, which is used for the trapdoor function and public key generation is deterministic, it could general predictable keys.
- There is a possibility that it is vulnerable to fault injection attacks through both instances. As such, this problem needs to be addressed under newer development runs of this scheme.
- Its security relies on ad-hoc assumptions, included with its added “Multi-Target Whipped MQ Problem, which lacks broad validation.
- Although it resists key leakage under well-chosen parameters, its theoretical guarantees remain uncertain for now.

# Limitations and Caveats of MAYO

- MAYO, while efficient and definitely faster & more optimized than something like RSA, still requires heavy computation and can occasionally fail to sign if its linear system lacks full rank.
- This is a rare but important design consideration, especially since it's still being tested.
- Also since the seed, which is used for the trapdoor function and public key generation is deterministic, it could general predictable keys.
- There is a possibility that it is vulnerable to fault injection attacks through both instances. As such, this problem needs to be addressed under newer development runs of this scheme.
- Its security relies on ad-hoc assumptions, included with its added “Multi-Target Whipped MQ Problem, which lacks broad validation.
- Although it resists key leakage under well-chosen parameters, its theoretical guarantees remain uncertain for now.

# Limitations and Caveats of MAYO

- MAYO, while efficient and definitely faster & more optimized than something like RSA, still requires heavy computation and can occasionally fail to sign if its linear system lacks full rank.
- This is a rare but important design consideration, especially since it's still being tested.
- Also since the seed, which is used for the trapdoor function and public key generation is deterministic, it could general predictable keys.
- There is a possibility that it is vulnerable to fault injection attacks through both instances. As such, this problem needs to be addressed under newer development runs of this scheme.
- Its security relies on ad-hoc assumptions, included with its added “Multi-Target Whipped MQ Problem, which lacks broad validation.
- Although it resists key leakage under well-chosen parameters, its theoretical guarantees remain uncertain for now.

# Limitations and Caveats of MAYO

- MAYO, while efficient and definitely faster & more optimized than something like RSA, still requires heavy computation and can occasionally fail to sign if its linear system lacks full rank.
- This is a rare but important design consideration, especially since it's still being tested.
- Also since the seed, which is used for the trapdoor function and public key generation is deterministic, it could general predictable keys.
- There is a possibility that it is vulnerable to fault injection attacks through both instances. As such, this problem needs to be addressed under newer development runs of this scheme.
- Its security relies on ad-hoc assumptions, included with its added “Multi-Target Whipped MQ Problem, which lacks broad validation.
- Although it resists key leakage under well-chosen parameters, its theoretical guarantees remain uncertain for now.

# Practical Key Generation Overview

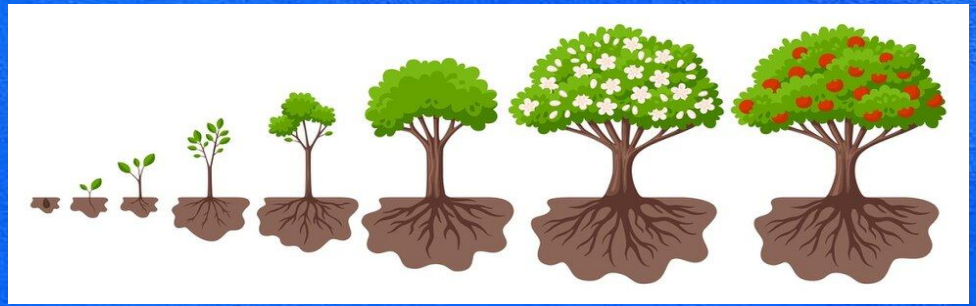
Start with a **small secret seed**

Seed expands into the full private key

Creates all needed parts: transformations, oil/vinegar split, whipping layers

Public key is made from this structure and then **compressed**

Results: **small keys, fast to make, easy to store**



# Mixed Modulus / Whipped Map Concept

Whipping layers **mix** the system so attackers can’t see the structure

More mixing = more security

Helps make the public key **smaller**

Uses GF(16), a small number system → **fast and lightweight**

Overall: quicker than many older MQ schemes

# Differences Between MAYO Variants

Each variant gives a

**different level of protection**

What changes:
– number of variables
– number of equations
– depth of whipping
– oil/vinegar balance


What stays the same:
– seed-based key generation
– compression
– whipped structure

Bigger variants = stronger security

Smaller variants = faster for low-power devices

No numbers here — just the concepts

# Key Generation Performance Insight

Seed expansion makes

key generation **very fast**

No need to build

huge systems manually

GF(16) math keeps everything light

Stays fast even at the highest levels

Great for phones, chips, and embedded devices

# PARAMETERS(WITHIN OUR SYSTEM)

The main parameters for the *real* scheme are:

- **q (Field Size):** The size of the number system. Real MAYO often uses $q=16$ (numbers from 0 to 15) because it's efficient.
- **n (Total Variables):** The total length of the full signature vector s.
- **m (Equations):** The number of equations in the public key, which matches the size of the hash output (e.g., 64 bytes).
- **k (Blocks):** The number of blocks the signature s is split into (e.g., k=8).
- **o(Oil Space):** The size of a secret part of the key. This is the "trapdoor" that allows Alice to sign, but which Bob (and any attackers) cannot find.

# Parameter Trade-offs and Practical Effects

- Oil dimension (o): In MAYO, o is relatively small compared to classical UOV. This is part of what lets the public key be compact. But smaller o means signing can be more expensive, because you're solving a linear system with fewer oil variables.
- Whipping parameter (k): This determines how many “base maps” are composed (“whipped”) to form the final public map. A larger k increases security but also increases the computational cost of signing.
- The two Level 1 variants reflect this trade-off: MAYO1 uses a smaller public key but pays in a larger signature size. MAYO2 uses a larger public key but has a more compact signature.
- For Level 3 and Level 5, the authors picked balanced parameter sets (MAYO3 and MAYO5) considering both security and performance.
- These trade-offs are not just small differences in numbers they directly influence how MAYO performs in real-world systems, especially when memory and bandwidth are constrained.

# Parameter Sets & Implementation

| Parameter set | MAYO_one | MAYO_two | MAYO_three | MAYO_five |
|---|---|---|---|---|
| security level | 1 | 1 | 3 | 5 |
| secret key size | 24 B | 24 B | 32 B | 40 B |
| public key size | 1420 B | 4912 B | 2986 B | 5554 B |
| signature size | 454 B | 186 B | 681 B | 964 B |

# Parameter Sets & Implementation

1. Security Levels: MAYO_one and MAYO_two both target security level 1, which is comparable to around 128-bit classical security.MAYO_three jumps to level 3, and MAYO_fivereaches level 5, which is the highest NIST category equivalent to 256-bit security. So as we move from left to right in the table, the security gets stronger.

2. Secret Key Sizes “Second, the secret key size. One thing that stands out in MAYO is how tiny the secret keys are: MAYO_one: 24 bytes, MAYO_two: 24 bytes, MAYO_three: 32 bytes and MAYO_five: 40 bytes

3. Public Key Sizes“Public key size is where we start to see trade-offs: MAYO_one: 1420 bytes, MAYO_two: 4912 bytes, MAYO_three: 2986 bytes and MAYO_five: 5554 bytes

4. Signature Sizes MAYO_one: 454 bytes, MAYO_two: 186 bytes, MAYO_three: 681 bytes and MAYO_five: 964 bytes

# Implementation Details

- The secret key is stored very compactly: only 24–40 bytes, depending on the variant. That's because the full secret key is generated from a small seed, not stored in its expanded form.
- The public key is also "seed + compressed data": the compact representation is used in storage or transmission, and then expanded when needed.
- Implementations exist in portable C, but also optimized for AVX2 and potentially for embedded platforms  this makes MAYO usable across very different hardware.
- The use of a deterministic seed-based expansion dramatically reduces memory and storage costs, which is especially valuable for IoT / embedded device use cases.

# Performance & Efficiency

| Variant | KeyGen / Expansion Cost | Signing Cost | Signature Size | Memory Footprint |
|---|---|---|---|---|
| **MAYO1** | Minimal overhead (seed expansion) | Medium (solving system) | 454 B | Low: compact seed + small map |
| **MAYO2** | Same seed-based overhead | Slightly faster due to different parameter trade-off | 186 B | Slightly more PK memory, but still efficient |
| **MAYO3 / MAYO5** | Higher expansion cost (but still efficient) | Signing cost increases with k and o | 681 B (MAYO3), 964 B (MAYO5) | Larger memory footprint, but manageable for many real-world systems |

# Performance & Efficiency

**Interpretation:**

- Key generation is lightweight since most of the “work” is just expanding a seed — not generating massive random matrices.
- Signing is more expensive than just a hash, but far more efficient than older multivariate schemes with large oil space.
- Verification (not shown in this table) is very efficient because it’s evaluating a quadratic map rather than solving a system.
- Memory remains low, especially when using the compact seed + compressed key representation: very practical for embedded systems.

# Performance & Efficiency

**Implementation(for experimentation):**

Each parameter set  has been tested on multiple CPUS. Specifically Intel and Apple Processors(which makes sense as those are the most popular as of now). It was implemented using the programming language C, which was then compiled on Ubuntu and ran 1000 benchmark runs. The following analysis will showcase the median of those runs.

# Performance and Efficiency (Haswell and AVX2)

## BEFORE (NO AVX Optimization)

On Intel Xeon E3-1225 v3 CPU (Haswell) at 3.20GHz for the optimized implementation:

| Scheme | KeyGen | ExpandSK + Sign | ExpandPK + Verify |
|---|---|---|---|
| MAYO_one | 1,052,858 | 3,171,356 | 530,101 |
| MAYO_two | 1,275,893 | 2,422,209 | 148,387 |
| MAYO_three | 3,230,269 | 10,725,576 | 1,313,842 |
| MAYO_five | 8,041,769 | 21,918,131 | 2,400,637 |

## AFTER (AVX Optimization)

On Intel Xeon E3-1225 v3 CPU (Haswell) at 3.20GHz for the AVX2 optimized implementation:

| Scheme | KeyGen | ExpandSK + Sign | ExpandPK + Verify |
|---|---|---|---|
| MAYO_one | 246,458 | 702,261 | 290,041 |
| MAYO_two | 153,420 | 375,493 | 96,606 |
| MAYO_three | 574,472 | 1,476,585 | 664,631 |
| MAYO_five | 1,338,275 | 3,475,547 | 1,488,808 |

The AVX2 optimized implementation on the Intel Haswell CPU is dramatically faster than the standard "Optimized implementation" on the same CPU, underscoring the necessity of using SIMD (Single Instruction, Multiple Data) instructions like AVX2 to accelerate the underlying mathematical operations. AVX2 As MAYO is meant to address evolving technologies in quantum and regular technological fields, AVX2 is meant to increase a computer's computational speed and efficiency by processing multiple data elements simultaneously.

# Performance and Efficiency (SKYLAKE)

## WITH NO AVX

On Intel Xeon E3-1260L v5 CPU (Skylake) at 2.90GHz for the optimized implementation:

| Scheme | KeyGen | ExpandSK + Sign | ExpandPK + Verify |
|---|---|---|---|
| MAYO_one | 899,214 | 2,797,221 | 441,935 |
| MAYO_two | 1,103,437 | 1,868,567 | 142,224 |
| MAYO_three | 2,998,299 | 8,615,942 | 1,119,046 |
| MAYO_five | 8,372,676 | 23,777,500 | 2,106,202 |

## WITH AVX

Intel Xeon E3-1260L v5 CPU (Skylake) at 2.90GHz for the optimized implementation:

| Scheme | KeyGen | ExpandSK + Sign | ExpandPK + Verify |
|---|---|---|---|
| MAYO_one | 202,026 | 574,530 | 247,169 |
| MAYO_two | 157,929 | 327,303 | 96,266 |
| MAYO_three | 473,424 | 1,260,260 | 589,375 |
| MAYO_five | 1,197,019 | 3,037,778 | 11,336,685 |

Just like before, the optimized AVX2 method is a lot fast except for mayo2's keygen. It seems our verification for mayo5 is also a caveat as well. AVX optimization speeds up both schemes, but it does not change the fundamental design trade-off which is mayo2 prioritizes fast verification at the cost of slow key generation, while mayo5 balances its speed in the opposite direction. Essentially, AVX2 makes what's great greater but what's worse becomes even worse as it ONLY prioritizes speed.

# Performance and Efficiency(Apple and ARM4)

ARM-4 processor (low powered) test:

| Scheme | KeyGen | ExpandSK + Sign | ExpandPK + Verify |
|---|---|---|---|
| MAYO_one | 9,572,510 | 16,911,188 | 11,242,036 |
| MAYO_two | 9,574,993 | 11,003,859 | 6,027,831 |
| MAYO_three | 27,170,014 | 45,428,633 | 28,446,049 |

Using the Apple M3 processor (high powered):

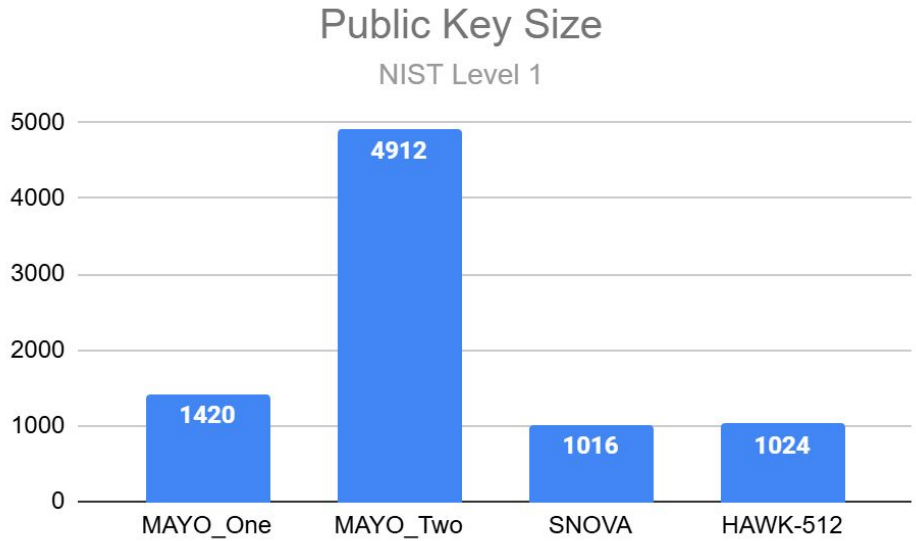
| Scheme | KeyGen | ExpandSK + Sign | ExpandPK + Verify |
|---|---|---|---|
| MAYO_one | 131,133 | 429,283 | 169,516 |
| MAYO_two | 125,437 | 279,380 | 72,482 |
| MAYO_three | 386,820 | 1,039,799 | 454,799 |
| MAYO_four | 895,085 | 2,217,257 | 948,399 |

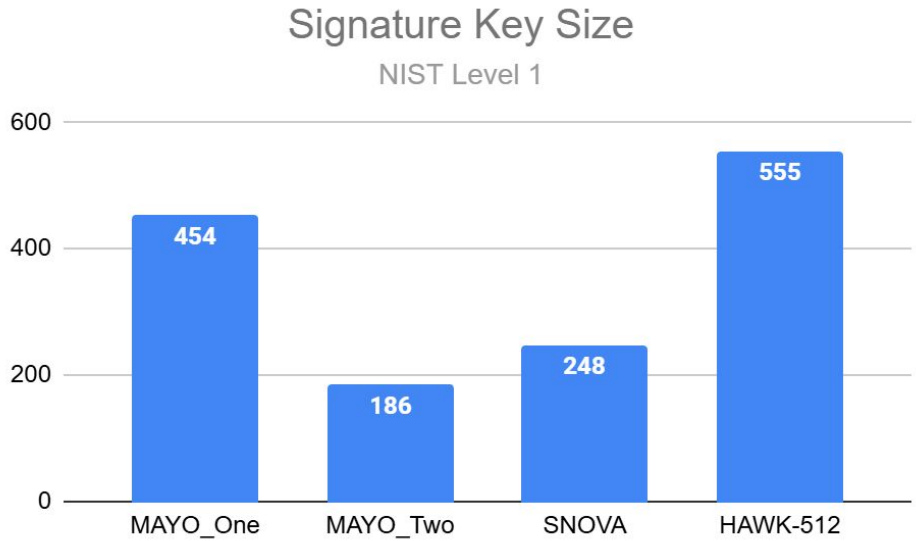
The Apple M-series 3 processor, shows the lowest cycle counts across all operations, making it the most efficient platform for running the MAYO scheme. This efficiency is characteristic of the ARM-based Apple Silicon design, which often features custom acceleration and tight integration of components.

# COMPARISONS

| Type | Sec. Lvl. | Key Gen. | Sign | Verify |
|---|---|---|---|---|
| MAYO [BCC+23] (default/pre-expanded) | | | | |
| $MAYO_1$ | 1 | $44k/44k$ | $218k/165k$ | $54k/31k$ |
| $MAYO_2$ | 1 | $86k/86k$ | $240k/142k$ | $47k/17k$ |
| $MAYO_3$ | 3 | $169k/169k$ | $719k/481k$ | $206k/131k$ |
| $MAYO_5$ | 5 | $370k/370k$ | $1\,244k/726k$ | $401k/221k$ |
| **Oil and Vinegar** [BCH+23] (pkc+skc/classic) | | | | |
| ovIp | 1 | $2\,316k/2\,341k$ | $1\,548k/79k$ | $168k/58k$ |
| ovIs | 1 | $3\,715k/3\,734k$ | $2\,063k/83k$ | $203k/46k$ |
| ovIII | 3 | $13\,168k/12\,832k$ | $8\,293k/243k$ | $679k/197k$ |
| ovV | 5 | $34\,989k/35\,792k$ | $18\,802k/462k$ | $1\,514k/364k$ |
| **TUOV** [DGG+] (pkc+skc/classic) | | | | |
| tuov-Ip | 1 | $3\,262k/5\,916k$ | $3\,639k/134k$ | $241k/56k$ |
| tuov-Is | 1 | $11\,797k/29\,323k$ | $19\,607k/130k$ | $273k/43k$ |
| tuov-III | 3 | $16\,237k/29\,815k$ | $18\,020k/354k$ | $1\,107k/191k$ |
| tuov-V | 5 | $38\,122k/68\,089k$ | $40\,046k/637k$ | $2\,947k/359k$ |
| **Dilithium** [LDK+20] | | | | |
| dilithium2 | 2 | $81k$ | $219k$ | $79k$ |
| dilithium3 | 3 | $137k$ | $355k$ | $129k$ |
| dilithium5 | 5 | $212k$ | $420k$ | $204k$ |
| **Falcon** [PFH+20] | | | | |
| falcon-512 | 1 | $20\,672k$ | $705k$ | $135k$ |
| falcon-1024 | 5 | $59\,019k$ | $1\,427k$ | $262k$ |
| **SPHINCS+** [HBD+20] | | | | |
| sha256-128f-simple | 1 | $618k$ | $14\,716k$ | $1\,269k$ |

# COMPARISONS



| Scheme | Public Key (bytes) | Signature Key (bytes) | Total (bytes) |
|---|---|---|---|
| *MAYO_One* | *1420* | *454* | *1874* |
| *MAYO_Two* | *4912* | *186* | *5098* |
| *SNOVA* | *1016* | *248* | *1264* |
| *HAWK-512* | *1024* | *555* | *1579* |

**Disadvantage of SNOVA:** It expands its public key into many quadratic polynomials, leading to the public key exceeding tens of kilobytes, making it less practical for constrained environments, such as IoT devices, embedded hardware, or bandwidth-limited systems.

**Disadvantage of HAWK-512:** It produces significantly larger signatures than competing post-quantum signature schemes, making it less suitable for constrained environments such as IoT devices or low-power embedded systems.

## Strengths

- Compact keys
- Fast verification
- Flexible parameters

## Weaknesses

- New assumption (Whipped MQ)
- Less studied than other NIST finalists

# Conclusion & Outlook

- MAYO is practical, lightweight, and quantum-safe
- Ideal for IoT/embedded
- Further cryptanalysis recommended

# Sources


- SNOVA Specification Document - Round 2
- HAWK v1.1 - Specification Document - Round 2
- Ward Beullens Fabio Campos Sof´ıa Celi Basil Hess Matthias J. Kannwischer
- https://pqmayo.org/params-times/
- https://www.researchgate.net/figure/MAYO-performance-in-CPU-cycles-using-AVX2-optimizations-in-comparison-with-other_tbl2_378951120

# Big round of applause for the Team behind our scheme!

Because *they* are the real heroes!

- **Ward Beullens**: (Cryptography Researcher at IBM Research)
- **Fabio Campos**: (Professor at Darmstadt University of Applied Sciences, Germany)
- **Sofía Celiasil Hess**: (Honorary Industrial Fellow in Cryptography for Privacy at the University of Bristol)
- **Matthias J. Kannwischer**: (Research Director at Chelpis Quantum Tech and Adjunct Assistant Professor at National Taiwan University)

# Thank you!

*And no, it's **not** a instrument!*